

1 Put a sheet IN → get a tool OUT

```
python main.py gui
python main.py grind your_sheet.xlsx
```

.xlsx .xlsm .csv .ipynb .py. Tidy sheets convert exactly; **messy ones use the AI Grinder** (scattered tables, multi-tab).

2 If it asks questions → answer

```
repository/<tool>/CLARIFICATIONS.md
```

e.g. "Cell H1 = 1.0 has no label — what is it?" Fix & re-grind. It never silently guesses.

3 See the result → the report

```
python main.py render <tool> --pdf
python main.py render <tool> --pdf --mode compact
```

→ repository/<tool>/reports/<tool>.pdf — branded A4, OK / NOT OK, bookmarks.

4 Print / share → normal PDF

Open the PDF, **Ctrl+P**. A machine-readable copy hides *inside* it:

```
reconstruct_from_pdf("file.pdf")
```

5 Change inputs → edit 1 file

```
repository/<tool>/<tool>.py
```

Edit numbers in `build_sheet()`, render again. Every value recomputes — never hand-typed.

6 Build a NEW tool from scratch

```
python main.py new "Punching Shear (EC2 6.4)"
python main.py register <slug>
python main.py render <slug> --pdf
```

7 Find a formula to reuse

```
python main.py atoms --search "crack width"
python main.py atoms --tree
```

Drill down country → code → chapter → clause, or call it: `custom.E_cm(f_cm=38)`.

8 See EVERYTHING (non-coders)

```
python main.py dashboard
```

→ repository/dashboard.html: what's hosted, recycled, who edited it, whole library — searchable.

Where things live: repository/ = your tools + reports (source of truth) · src/library/ = shared formulas (atoms) · docs/ = deeper guides & PITFALLS. **Full run once:** `python main.py demo`. **Does it exist already?** `python main.py ask "what you want"`.